

Relational Data Model and Relational Algebra

Relational Algebra

- Definition of relational Algebra
- Purpose of Relational Algebra
- Relational algebra Operations
- Join
- Types of join
- Aggregate function
- ER Diagram to Relational Model
- More Example

Relational Algebra

- **Definition:**
 - Relational algebra is a procedural query language.
 - It consists of a set of operations that takes one or more relations as input and produce a new relation as their result.

Relations, Entities, Tables

Relational model	E/R Diagram	SQL
Relation Tuple Attribute Foreign Key Primary Key	Entity Instance Attribute M:1 Relationship	Table Row Column or Field Foreign Key Primary Key

Query language

Query languages: Allow manipulation and retrieval of data from a database.

These are categorized as being either procedural or non procedural.

Procedural:

The user instructs the system to perform a sequence of operations on the database to compute the desired result.

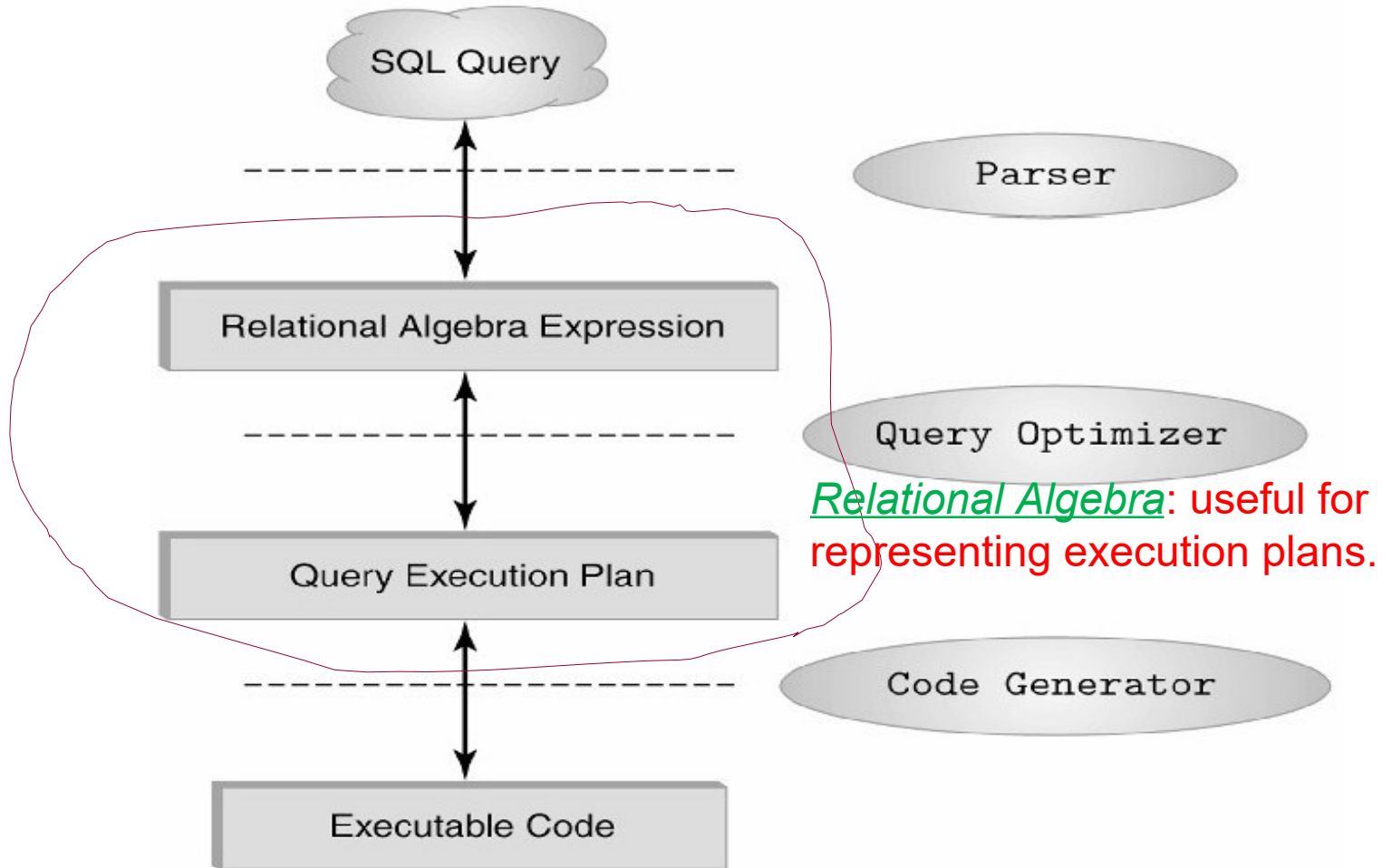
PL/SQL is Procedural.

Non-Procedural:

In contrast the non-procedural languages user specify what they want and do not specify how to do.

SQL is Non-Procedural.

Role of Relational algebra in DBMS



- The operations of relational algebra can be divided into two groups:
 - Set operations : each relation is defined to be set of tuples in relational model.
 - UNION
 - INTERSECTION
 - SET DIFFERENCE
 - CARTESIAN PRODUCT
 - Other group consists of operations developed for relational databases:
 - SELECT
 - PROJECT
 - JOIN

Unary Relational Operations

Unary operators : operators can be applied to a single relations

- SELECT (symbol: σ (sigma))
- PROJECT (symbol: π (pi))
- RENAME (symbol: ρ (rho))

Binary operators : Needs more than relations as input and produce a new relation

- Cartesian product: (Symbol : $\times$)
- Join (symbol: $\bowtie$)

Unary Relational Operations

- The SELECT Operation: selects a subset of the tuples from a relation that satisfy a selection condition.
- **Notation:** $\sigma_p(r)$ σ (sigma) is used to denote the SELECT operator, p is selection condition (boolean expression)
- Multiple condition can be connected by $\wedge$ (and), $\vee$ (or), $\neg$ (not)
- Student

Sid	Sname	Rating	Age
11	Ramesh	9	25
12	Mamatha	8	27
13	Asha	7	30
14	Akash	9	28

$\sigma_{\text{rating}>8}(\text{Student})$

Sid	Sname	Rating	Age
11	Ramesh	9	25
14	Akash	9	28

- $\sigma_{\text{rating}>8 \wedge \text{age}<30}(\text{Student})$

Sid	Sname	Rating	Age
11	Ramesh	9	25
14	Akash	9	28

- The PROJECT Operation: selects specified columns from the table and discards the other columns.
- **Notation:** $\Pi_{A_1, A_2, \dots, A_k}(r)$
- where A_1, A_2 are the desired list of attributes and r is a relation name.
- Stud

Sid	Sname	Rating	Age
11	Ramesh	9	25
12	Mamatha	8	27
13	Asha	7	30
14	Akash	9	28

$\pi_{\text{age}}(\text{Stud})$

Age
25
27
30
28

$\pi_{\text{sid,age}}(\text{Stud})$

Sid	Age
11	25
12	27
13	30
14	28

Selection and Projection

- Select - Selects rows that satisfy *selection condition*.
- Project - selects specified columns

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

$\sigma_{rating > 8}(S2)$

sname	rating
yuppy	9
rusty	10

$\pi_{sname, rating}(\sigma_{rating > 8}(S2))$

Relational Algebra Operations from Set Theory

(Union, Intersection and Minus Operation)

- **Union**: denoted by $R \cup S$, is a relation that includes all tuples that are either in R or in S or in both R and S . Duplicate tuples are eliminated. Notation: $r \cup s$
- **Defined as:**
$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$
- For $r \cup s$ to be valid.
 1. r, s must have same number of attributes
 2. The attribute domains must be compatible

Relational Algebra Operations from Set Theory

(Union, Intersection and Minus Operation)

- **Intersection:** denoted by $R \cap S$, is a relation that includes all tuples that are in both R and S .
- Notation $r \cap s$
- **Defined as:**
$$r \cap s = \{t \mid t \in r \text{ and } t \in s\}$$
- Set intersection must be taken between compatible relations.
 - r and s must have the same attribute
 - attribute domains of r and s must be compatible

Relational Algebra Operations from Set Theory

(Union, Intersection and Minus Operation)

- **Set Difference(or Minus)**: denoted by $R-S$, is a relation that includes all tuples that are in R but not in S .

Notation: $r - s$

- **Defined as:**

$$r - s = \{t \mid t \in r \text{ and } t \notin s\}$$

- Set differences must be taken between compatible relations.
 - r and s must have the same attributes
 - attribute domains of r and s must be compatible

- S1

Sid	Sname	Rating	Age
11	Ramesh	9	25
12	Mamatha	8	27
13	Asha	7	30
14	Akash	9	28

S2

Sid	Sname	Rating	Age
11	Ramesh	9	25
19	Kavitha	10	27
13	Asha	7	30
16	Bharath	9	31

- S1 $\cup$ S2

Sid	Sname	Rating	Age
11	Ramesh	9	25
12	Mamatha	8	27
13	Asha	7	30
14	Akash	9	28
19	Kavitha	10	27
16	Bharath	9	31

S1 $\cap$ S2

Sid	Sname	Rating	Age
11	Ramesh	9	25
13	Asha	7	30

S1 – S2

Sid	Sname	Rating	Age
12	Mamatha	8	27
14	Akash	9	28

Set Difference

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S1

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

S2

sid	sname	rating	age
22	dustin	7	45.0

$S1 - S2$

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
44	guppy	5	35.0

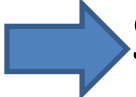
$S2 - S1$

S1-S2 not equal to S2-S1

- Used to rename a relation, or columns in the relation.

Boats (bid, name, color)  Ship (bid, name, color)

ρ_{Ship} (Boats)

Boats (bid, name, color)  Ship (sid, sname, shade)

ρ_{Ship} (sid, sname, shade) (Boats)

Binary Relational Operations

- Join Operation: used to combine information from two or more relations. ($\bowtie$)
 - Rows of 2 table are combined based on the given column(s) values.
 - The tables being joined must have one common column
 - Generic Join operator $\bowtie$

- Types of join
- Cross join
- Inner join – only those tuples that satisfy the matching criteria are included, while the rest are excluded. Possibility of loss of information.
 - Equi-join
 - Natural join
- Outer join – along with tuples that satisfy the matching criteria, also include some or all tuples that do not match the criteria. No loss of information
 - Left outer join
 - Right outer join
 - Full outer join
- Self-join- used to join a table to itself, as if the tables were two tables.

Binary Relational Operations

- The Cartesian Product (or Cross Product) Operation : Also called **CROSS JOIN** – denoted by X.
- Used to combine tuples from two relations in a combinatorial fashion.
- Each and every column of the first table will combine with each and every column of the second table.

• R

a
b

S

1
2
3

R x S

a	1
a	2
a	3
b	1
b	2
b	3

R1

Cross Product Example

<u>sid</u>	sname	rating	age
22	xxx	7	45.0
31	yyy	8	55.5
58	zzz	10	35.0

S1

<u>sid</u>	<u>bid</u>	<u>day</u>
22	101	10/10/96
58	103	11/12/96

R1 X S1 =

(sid)	sname	rating	age	(sid)	bid	day
22	xxx	7	45.0	22	101	10/10/96
22	xxx	7	45.0	58	103	11/12/96
31	yyy	8	55.5	22	101	10/10/96
31	yyy	8	55.5	58	103	11/12/96
58	zzz	10	35.0	22	101	10/10/96
58	zzz	10	35.0	58	103	11/12/96

Cross Product Example

- Account

Ano	Bname	Balanca
101	JP Naga	10000
102	BTM	20000

Depositor

Dname	Ano
Sumith	101
Ravi	102
Kiran	103

- Find customer name having balance atleast <15000

Acc.Ano	Bname	Balanca	Dname	d.Ano
101	JP Naga	10000	Sumith	101
101	JP Naga	10000	Ravi	102
101	JP Naga	10000	Kiran	103
102	BTM	20000	Sumith	101
102	BTM	20000	Ravi	102
102	BTM	20000	Kiran	103

$$\pi_{Dname}(\sigma_{a.ano=D.ano}^{bal < 15000}(Account \times Deposit))$$

- EQUI – JOIN:(Inner Join)

Links tables by selecting rows with common values for common attribute(s) & Does not include rows that are unmatched

Eg: EMP

NAME	OFFICE	DEPT	SALARY
Sumith	400	CS	25000
Ravi	220	Econ	30000
Kiran	160	Elec	35000
Sathish	420	Econ	40000

DEPART

DEPT	MAIN OFFICE	PHONE
CS	404	2436789
Econ	200	2378658
Fin	501	2356297
Hist	100	2745897

- Eg: EMP DEPART
emp.dept=depart.dept

NAME	OFFICE	emp.DEPT	SALARY	Depart.DEPT	MAIN OFFICE	PHONE
Sumith	400	CS	25000	CS	404	2436789
Ravi	220	Econ	30000	Econ	200	2378658
Sathish	420	Econ	40000	Econ	200	2378658

- **NATURAL JOIN:** Links tables by selecting rows with common values in common attribute(s) and Does not include rows that are unmatched
- similar to EQUI JOIN but **natural join eliminates duplicate columns** for the joining columns.

NAME	OFFICE	DEPT	SALARY	MAIN OFFICE	PHONE
Sumith	400	CS	25000	404	2436789
Ravi	220	Econ	30000	200	2378658
Sathish	420	Econ	40000	200	2378658

- NON-EQUI JOIN: joins two or more tables based on non equaling conditions.
- Eg: EMP  emp.dept!=depart.dept DEPART

NAME	OFFICE	DEPT	SALARY	DEPT	MAIN OFFICE	PHONE
Sumith	400	CS	25000	Econ	200	2378658
Sumith	400	CS	25000	Fin	501	2356297
Sumith	400	CS	25000	Hist	100	2745897
Ravi	220	Econ	30000	CS	404	2436789
Ravi	220	Econ	30000	Fin	501	2356297
Ravi	220	Econ	30000	Hist	100	2745897
Kiran	160	Elec	35000	CS	404	2436789
Kiran	160	Elec	35000	Econ	200	2378658
Kiran	160	Elec	35000	Fin	501	2356297
Kiran	160	Elec	35000	Hist	100	2745897
Sathish	420	Econ	40000	CS	404	2436789
Sathish	420	Econ	40000	Fin	501	2356297
Sathish	420	Econ	40000	Hist	100	2745897

- OUTER JOINS: 3 types

- **Left outer join** $\bowtie \leftarrow$: includes all tuples in the left hand relation and only those matching tuples from the right hand relation.
- **Right outer join** $\rightarrow \bowtie$: includes all tuples in the right hand relation and only those matching tuples from the left hand relation.
- **Full outer join** $\bowtie$: includes all tuples in the left hand relation and from the right hand relation.



- Eg: student

SNO	NAME	AGE
S1	N1	21
S2	N2	22
S3	N3	23
S4	N4	24

Course

Course

Sno	Cname	Fees
S1	C1	1000
S3	C3	3000
S5	C5	5000



Left outer join
from the r

SNO	NAME	AGE	Cname	Fees
S1	N1	21	C1	1000
S2	N2	22	Null	Null
S3	N3	23	C3	3000
S4	N4	24	Null	Null

matching tuples

- Eg: student

Course

SNO	NAME	AGE
S1	N1	21
S2	N2	22
S3	N3	23
S4	N4	24

Sno	Cname	Fees
S1	C1	1000
S3	C3	3000
S5	C5	5000

Course



Right outer join
from the left

SNO	NAME	AGE	Cname	Fees
S1	N1	21	C1	1000
S3	N3	23	C3	3000
S5	Null	Null	C5	5000

matching tuples

- Eg: student

Course

SNO	NAME	AGE
S1	N1	21
S2	N2	22
S3	N3	23
S4	N4	24

Sno	Cname	Fees
S1	C1	1000
S3	C3	3000
S5	C5	5000

Course



Full outer join
relation.

SNO	NAME	AGE	Cname	Fees
S1	N1	21	C1	1000
S2	N2	22	Null	Null
S3	N3	23	C3	3000
S4	N4	24	Null	Null
S5	Null	Null	C5	5000

from the right hand

- Division Operator:

- For A/B attributes of B are subset of attributes of A.
- The relation returned by division operator will have
attributes = (All attributes of A – All Attributes of B)
- The relation returned by division operator **will return those tuples from relation A which are associated to every B's tuple.**

- Eg of Division : A/B

A

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

B1

pno
p2

B2

pno
p2
p4

B3

pno
p1
p2
p4

A/B1

sno
s1
s2
s3
s4

A/B2

sno
s1
s4

A/B3

sno
s1

$$B2 = \{S1, S2, S3, S4\} \cap \{S1, S4\}$$

$$B3 = P1 \rightarrow \{S1, S2\} \cap \{S1, S2, S3, S4\} \cap \{S1, S4\}$$

Aggregate Functions

- **Aggregation function (ζ)** takes a collection of values and returns a single value as a result.

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

- Eg:

Name	Office	Dept	Salary
Mithali	400	Cs	20000
Kavitha	220	Cs	45000
Sanjay	160	Econ	30000
Varun	420	Econ	55000
Asha	500	Fin	65000

1. Find the minimum salary: $\zeta_{\min(\text{salary})}\text{EMP}$

Min(Salary)
20000

2. Find the maximum salary: $\zeta_{\max(\text{salary})}\text{EMP}$

Max(Salary)
65000

3. Find the average salary: $\zeta_{\text{avg}(\text{salary})}\text{EMP}$

Avg(Salary)
43000

4. Count the no. of employees in the cs dept: $\zeta_{\text{count}(\text{name})}(\sigma_{\text{dept}=\text{'Cs'}}(\text{EMP}))$

Count(name)
2

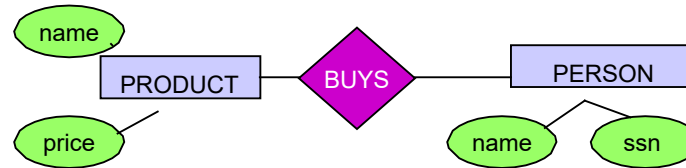
5. Find the total payroll for the economics dept:

$\zeta_{\text{sum}(\text{salary})}(\sigma_{\text{dept}=\text{'Econ'}}(\text{EMP}))$

Sum(Salary)
85000

ER diagram to Relational mapping

Conceptual
Model:



Relational
Model:

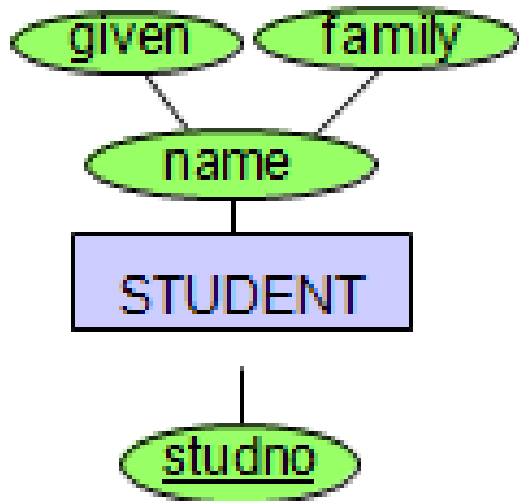
ER diagram to Relational mapping

7-Step Process:

1. Map Regular Entity Types
2. Map Weak Entity Types
3. Map Binary 1:1 Relation Types
4. Map Binary 1:N Relationship Types.
5. Map Binary M:N Relationship Types.
6. Map Multivalued attributes.
7. Map N-ary Relationship Types.

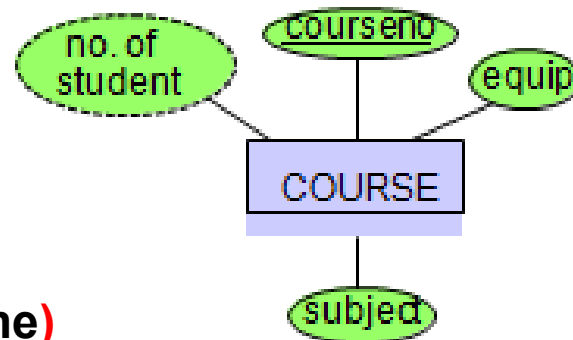
Mapping Entity Types to Relations

- For every *entity type* create a *relation*
- Every *atomic attribute* of the entity type becomes a *relation attribute*
- *For Composite attributes*: include *all the atomic attributes*
- *Derived attributes* are not included
- Attributes of the *entity key* make up the *primary key* of the relation



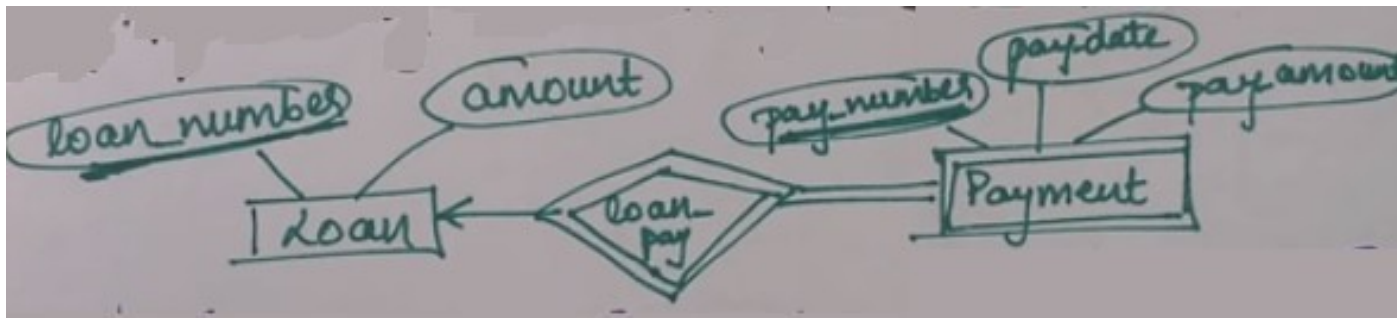
STUDENT (studno, givenname, familyname)

COURSE (courseno, subject, equip)



Mapping Weak Entity Types to Relations

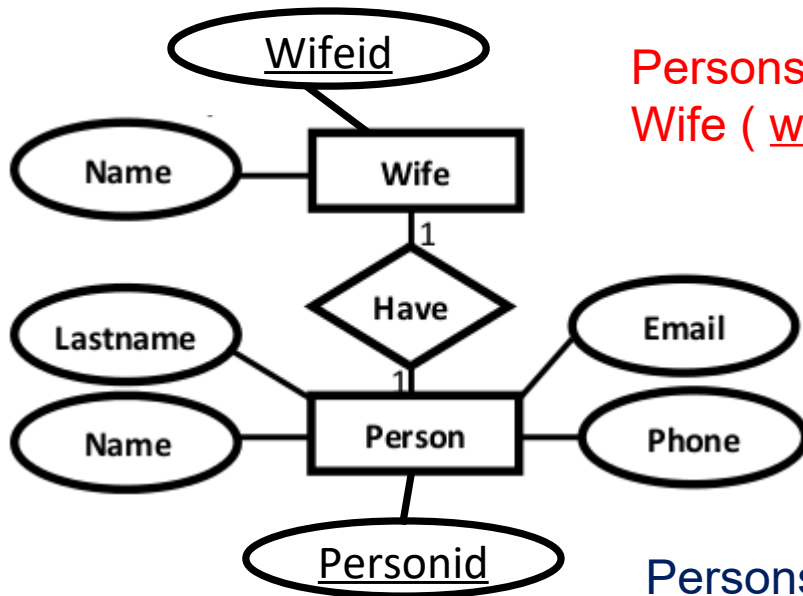
- Create a Relation S for weak entity set.
- Add all its attributes to relation S as field.
- Add the primary key of identifying entity set R to Relation S and Declare all foreign key constraints.



- Loan (Loan_num, amount)
- Payment(Loan_num, pay_number, paydate, payamt)

Mapping 1 to 1 Relationship

- identify the two relations correspond to the entity types participating in the relationship.
- Place the primary key of anyone relation as the foreign key of other relation.



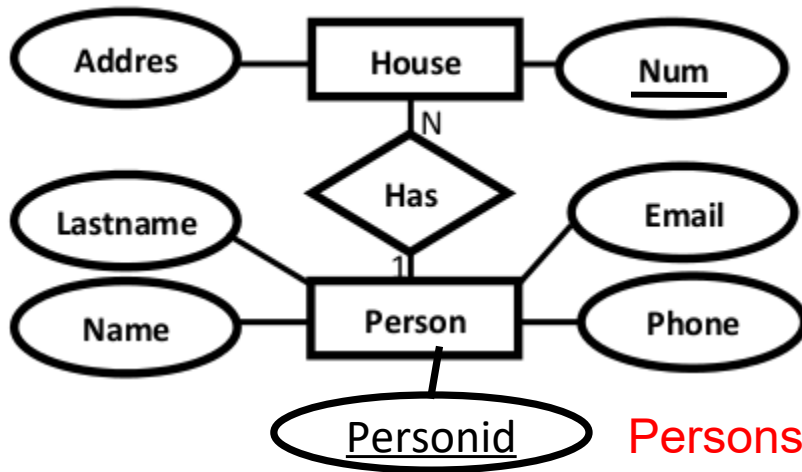
Persons(personid , name, lastname, email , **wifeid**)
Wife (wifeid , name)

Or

Persons(personid , name, lastname, email)
Wife (wifeid , name , **personid**)

Mapping 1 to N or N to 1 Relationship

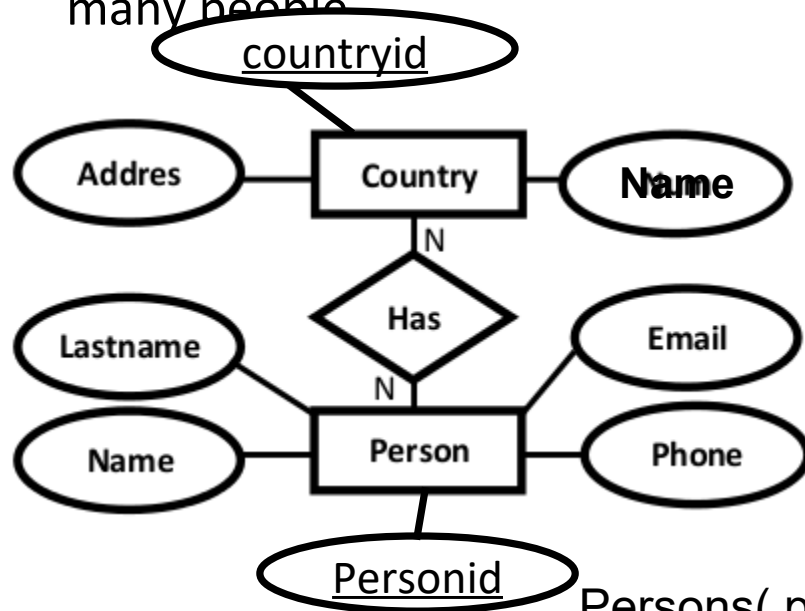
- let T and S be the entity sets with (1:N) where S is having total participation in the relationship
- add primary key of T into S as foreign key.
- (Primary key of 1 side to N side as foreign key)



Persons(personid , name, lastname, email)
House (Num , address, personid)

Mapping M:N Relationship

- Create a third relation containing the primary key of both entity sets and descriptive attribute (if any)
- The Person can live or work in many countries. Also, a country can have many people



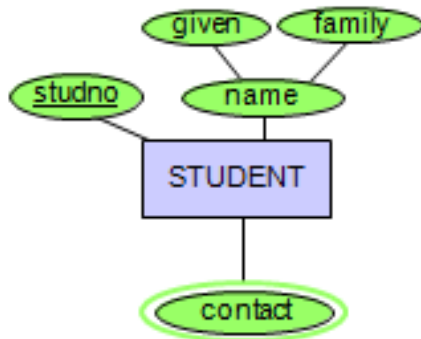
Persons(personid , name, lastname, email,Phone)

Countries (countryid , name, Address)

HasRelat (hasrelatid , **personid** , **countryid**)

Mapping Multivalued attribute

- For each multivalued attribute create a new relation T.
- Add primary key of each entity set as foreign key of Relation T.
- The foreign key attribute and multivalued attribute become composite key.



STUDENT

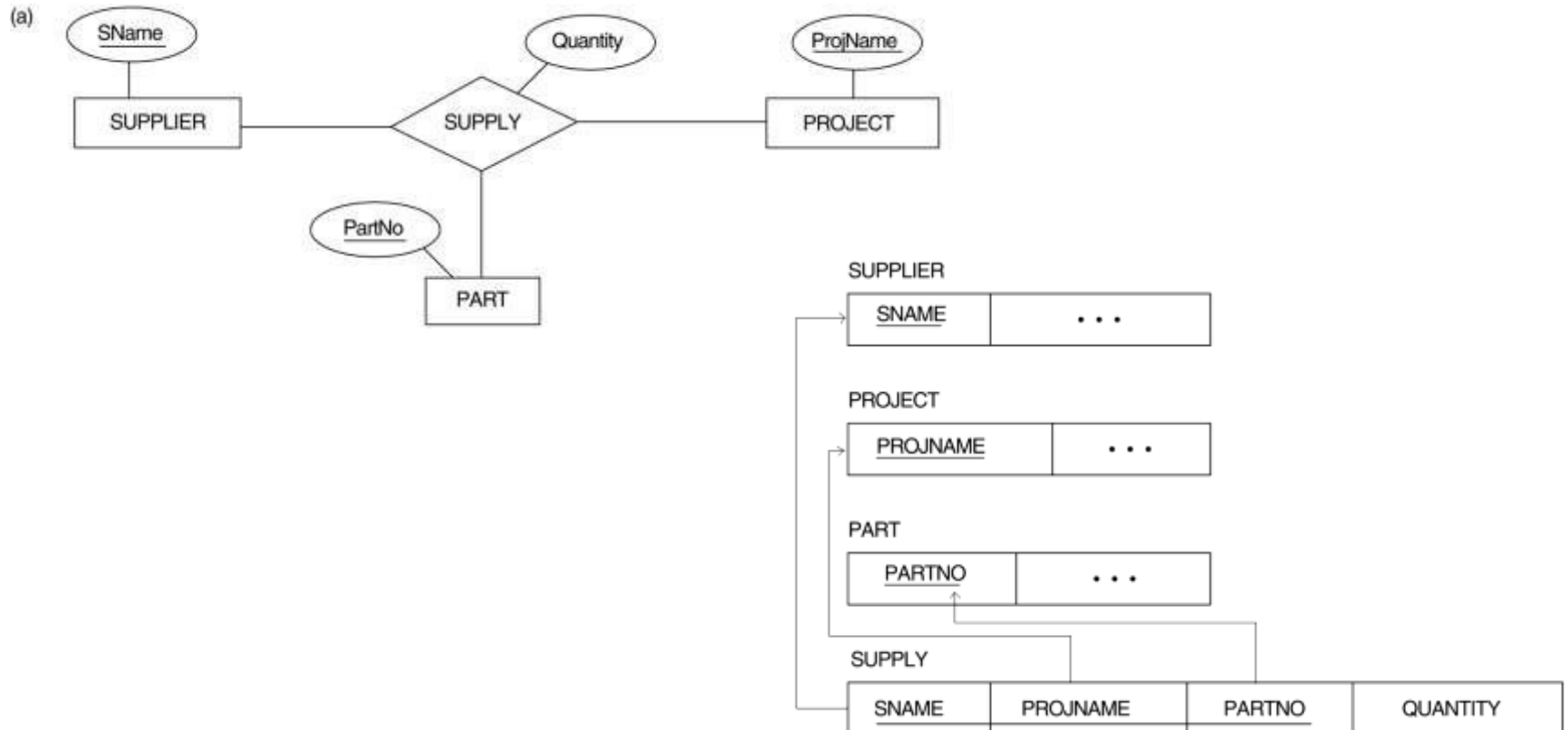
<u>studno</u>	given	family
s1	fred	jones
s2	mary	brown

STUDENT_CONTACT

<u>studno</u>	<u>contact</u>
s1	Mr. Jones
s1	Mrs Jones
s2	Bill Brown
s2	Mrs Jones
s2	Billy-Jo Woods

Mapping N-ary relationship

- For each N-ary relationship, create a new relation.
- New relation will contain the primary key of all the entity sets forming the N-ary relationship



Some more example Queries

Consider the schema **depositor** (Cust_name, acc_number)

Borrower (cust_name, loan_number) **loan**(branch_name, loan_number, amount)

- Find all loans of over \$1200

$$\sigma_{\text{amount} > 1200} (\text{loan})$$

- Find the loan number for each loan of an amount greater than \$1200

$$\Pi_{\text{loan_number}} (\sigma_{\text{amount} > 1200} (\text{loan}))$$

- Find the names of all customers who have a loan, an account, or both, from the bank

$$\Pi_{\text{customer_name}} (\text{borrower}) \cup \Pi_{\text{customer_name}} (\text{depositor})$$

- Find the names of all customers who have a loan and an account at bank.

$$\Pi_{\text{customer_name}} (\text{borrower}) \cap \Pi_{\text{customer_name}} (\text{depositor})$$

Example Queries

Consider the schema

depositor (Cust_name, acc_number)
Borrower (cust_name, loan_number) **loan**(branch_name, loan_number, amount)

- Find the names of all customers who have a loan at the Perryridge branch.

$$\Pi_{\text{customer_name}} \left(\sigma_{\text{branch_name} = \text{"Perryridge"}} \left(\sigma_{\text{borrower.loan_no} = \text{loan.loan_no}} (\text{borrower} \times \text{loan}) \right) \right)$$

- `lives(person-name,street,city)`
- `works(person-name, company-name,salary)`
- `located-in(company-name,city)`
- `manages(person-name,manager-name)`

Find the name of all employees (i.e., persons) who work for the City Bank company (which is a specific company in the database).

$$\pi_{person-name}(\sigma_{cname='City Bank'}(works))$$

- `lives(person-name, street,city)`
- `works(person-name, company-name, salary)`
- `located-in(company-name, city)`
- `manages(person-name,manager-name)`

Find the name and city of all employees who work for City Bank.

$$\pi_{lives.pname, lives.city}(\sigma((cname='City Bank') \wedge (works \times lives)) \\ (lives.pname=works.pname)).$$

- Similar to previous query, except we have to access the lives table to extract the city of the employee. The join condition is the same person name in the two tables Lives and Works.

- `lives(person-name, street, city)`
 - `works(person-name, company-name, salary)`
 - `located-in(company-name, city)`
 - `manages(person-name, manager-name)`
-
- Find all persons who do not work for City Bank.

$$\pi_{pname}(works - (\sigma_{cname='City Bank'}(works)))$$

- **lives(person-name, street, city)**
- works(person-name, company-name, salary)
- **located-in(company-name, city)**
- manages(person-name, manager-name)

Find all employees who live in the same city as the company they work for.

For this query we need to access the lives table to get city of the employee and the located-in table to get city of the company; plus the works table to associate employee with their company. The selection condition is then that the two cities are the same.

$$\pi_{lives.pname} \left(\sigma_{((locatedin.cname=works.cname) \wedge (located-in.city=lives.city) \wedge (lives.pname=works.pname))} (works \times lives \times locatedin) \right)$$

Questions

- Explain select and Project operations
- Explain union, intersect and minus operations.
- Explain division operator.
- Explain join and types of join
- Write short note on Natural join with example
- Explain Types of Outer join
- List the steps to convert ER diagram to Relational model